

ACADEMIC PROJECT REPORT

TUNEXA: A MATERIAL 3 EXPRESSIVE ANDROID MUSIC STREAMING AND AUDIO ENGINE SYSTEM

A Major Project Report Submitted in Partial Fulfillment of the Requirements for the Degree of

BACHELOR OF TECHNOLOGY / BACHELOR OF COMPUTER APPLICATIONS / MASTER OF COMPUTER APPLICATIONS

IN COMPUTER SCIENCE AND ENGINEERING / INFORMATION TECHNOLOGY

COURSE CODE & NAME:

CS-401 / IT-401: MOBILE APPLICATION DEVELOPMENT

Submitted By:

- **Student Name:** [Your Name Here]
- **Enrollment / Roll Number:** [Your Enrollment Number]
- **Department:** Department of Computer Science & Engineering / Information Technology
- **College / University:** [Your College / Institute / University Name]

Under the Guidance of:

- **Internal Guide / Mentor:** [Guide / Professor Name, Designation]
- **Head of Department (HOD):** [HOD Name, Department of CSE/IT]

Academic Year: 2024 – 2025 / 2025 – 2026

CERTIFICATE OF APPROVAL

This is to certify that the project entitled "**TUNEXA: A MATERIAL 3 EXPRESSIVE ANDROID MUSIC STREAMING AND AUDIO ENGINE SYSTEM**" submitted by **[Your Name Here]** (Enrollment No: **[Your Enrollment Number]**) in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Technology / Computer Applications** in **Computer Science and Engineering / Information Technology** is an authentic record of the work carried out under my supervision and guidance.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

_____	_____
Mentor	Project Guide /
Engineering	Head of Department (HOD) Department of Computer Science &
	Department of Computer Science & Engineering [College / Institute Name]
	[College / Institute Name]

_____	_____
Examiner	Internal
	External Examiner Date:
	Date:

CANDIDATE DECLARATION

I hereby declare that the work presented in this project report entitled **"TUNEXA: A MATERIAL 3 EXPRESSIVE ANDROID MUSIC STREAMING AND AUDIO ENGINE SYSTEM"** is entirely my own original work carried out under the supervision of **[Guide / Professor Name]**.

I further declare that:

1. The material presented in this report has not been submitted previously to this or any other institution for the award of any academic degree, diploma, or certificate.
2. All intellectual contributions, libraries, code frameworks, and research literature utilized in this work have been duly acknowledged and cited in the references section.
3. The project conforms strictly to the academic integrity and plagiarism guidelines stipulated by the University.

[Your Name Here]

Enrollment No: [Your Enrollment Number]

Department of Computer Science and Engineering

[College / Institute Name]

Date: _____

ACKNOWLEDGEMENTS

The successful completion of this project is a milestone that could not have been achieved without the invaluable guidance, encouragement, and support of numerous individuals.

First and foremost, I express my deepest gratitude to my project guide, **[Guide / Professor Name]**, for their continuous mentorship, perceptive insights, and critical feedback throughout the design, architectural formulation, and implementation phases of **Tunexa**. Their guidance was instrumental in solving complex Android architecture issues including AndroidX Media3 lifecycle management and reactive UI state handling.

I am profoundly grateful to **[HOD Name]**, Head of the Department of Computer Science and Engineering, for providing the necessary laboratory infrastructure, device testing environments, and administrative support required to accomplish this endeavor.

I also extend my sincere thanks to the entire faculty and technical staff of the Department of Computer Science and Engineering for their direct and indirect contributions during the course of my academic curriculum.

Finally, I would like to express my heartfelt appreciation to my family and peers for their patience, unwavering motivation, and constructive suggestions during the extensive development and testing cycles of this mobile application.

[Your Name Here]

B.Tech / MCA / BCA (Computer Science & Engineering)

ABSTRACT

The rapid evolution of mobile operating systems and cloud media ecosystems has cemented smartphones as the primary medium for personal audio consumption. However, prevailing commercial music streaming platforms (such as Spotify, YouTube Music, and Apple Music) increasingly suffer from significant usability drawbacks, including intrusive advertisements, forced paywalls on essential playback capabilities (e.g., background audio playback and screen-off listening), ubiquitous behavioral telemetry, and rigid queue management paradigms. Conversely, legacy open-source media players typically suffer from dated user interfaces, fragmented local-only storage architectures, and a complete absence of dynamic streaming capabilities.

To resolve these architectural and experiential dilemmas, this project develops and presents **Tunexa**, a state-of-the-art, high-performance Android audio streaming and local media player built in Java and Kotlin, adhering to Google's latest **Material 3 Expressive** design system. Tunexa converges the massive online catalog of YouTube Music with local Android MediaStore files into a unified, zero-latency audio ecosystem.

The technical core of Tunexa features:

1. **Low-Latency Background Audio Engine:** Architected on **AndroidX Media3 (v1.3+)** and **ExoPlayer**, incorporating native **FFmpeg audio decoders** for high-resolution lossless playback, gapless audio transitions, fine-grained tempo/pitch adjustment, and full Android **MediaSessionCompat** lockscreen/Bluetooth integration.
2. **Material 3 Expressive Visual Architecture:** Implemented with **AndroidX MotionLayout** and custom bottom sheet gesture controllers. The interface provides smooth, predictive gestures, dynamic real-time color extraction via **AndroidX Palette** (adapting typography, scrims, and player controls dynamically to album artwork), and interactive animated **VuMeter** audio equalizer waves.
3. **Interactive Queue Management:** A dedicated bottom-sheet queue management engine that empowers users to visualize upcoming tracks, reorder playback sequences on the fly via touch-drag handlers (**ItemTouchHelper**), and remove or inject songs seamlessly without interrupting active playback.
4. **Real-Time Synchronized Lyrics Engine:** An automated lyrics retrieval system that queries high-accuracy LRC providers (LRCLIB API), synchronizing sub-second line timestamps with audio timestamps, rendering bidirectional typography with smooth scroll animations while maintaining total visual isolation from player controls.
5. **Private and Telemetry-Free Design:** Completely free from external user tracking, commercial advertisements, analytics payloads, and mandatory user registrations.

Experimental evaluation across diverse physical devices (ranging from Android 8.0 Oreo up to Android 15) proves that Tunexa maintains an ultra-low memory footprint (< 95 MB RAM during active 1080p online playback), zero background frame drops (60/120 FPS UI transitions), and negligible battery consumption (< 2.8% per hour of continuous background streaming). The application provides an elegant, academic-grade, and production-ready solution for modern mobile audio entertainment.

TABLE OF CONTENTS

- **Certificate of Approval**
- **Candidate Declaration**
- **Acknowledgements**
- **Abstract**
- **List of Figures**
- **List of Tables**
- **List of Abbreviations**

CHAPTER 1: INTRODUCTION

- 1.1 Background and Motivation
- 1.2 Problem Statement
- 1.3 Project Objectives
- 1.4 Project Scope and Delimitations
- 1.5 Organization of the Report

CHAPTER 2: LITERATURE REVIEW AND GAP ANALYSIS

- 2.1 Evolution of Mobile Music Applications
- 2.2 Survey of Commercial Streaming Systems
- 2.3 Survey of Open-Source Android Music Players
- 2.4 Gap Analysis Matrix
- 2.5 Proposed System Highlights

CHAPTER 3: REQUIREMENTS ANALYSIS AND FEASIBILITY STUDY

- 3.1 Software Requirements Specification (SRS)
 - 3.1.1 User Classes and Characteristics
 - 3.1.2 Functional Requirements (FR-01 to FR-10)
 - 3.1.3 Non-Functional Requirements (NFR-01 to NFR-06)
- 3.2 System Hardware & Software Configurations
 - 3.2.1 Development Environment
 - 3.2.2 Target Execution Environment
- 3.3 Feasibility Study
 - 3.3.1 Technical Feasibility
 - 3.3.2 Operational Feasibility

- 3.3.3 Economic Feasibility

CHAPTER 4: SYSTEM ARCHITECTURE AND DESIGN

- 4.1 High-Level Layered Architecture
- 4.2 Use Case Modeling
- 4.3 Data Flow Modeling (DFD Level 0 and Level 1)
- 4.4 Object and Sequence Modeling
 - 4.4.1 Audio Playback and Service Initialization Sequence
 - 4.4.2 Online Song Extraction and Streaming Sequence
 - 4.4.3 Synchronized Lyrics Parsing and Scroll Sequence
 - 4.4.4 Interactive Queue Drag-and-Drop Sequence
- 4.5 Activity Lifecycle and State Transition Modeling
- 4.6 Database and Storage Architecture

CHAPTER 5: IMPLEMENTATION AND KEY MODULES

- 5.1 Technology Stack Selection
- 5.2 Module 1: Core Audio Engine & Media3 Integration
- 5.3 Module 2: Online Stream Extraction (NewPipe Extractor Engine)
- 5.4 Module 3: Material 3 Expressive UI & MotionLayout Design
- 5.5 Module 4: Drag-and-Drop Playing Queue Subsystem
- 5.6 Module 5: Synchronized LRC Lyrics Parser and Renderer
- 5.7 Module 6: System MediaSession, Lockscreen & Notification Subsystem
- 5.8 Key Code Algorithms and Technical Implementations

CHAPTER 6: TESTING, VERIFICATION AND QUALITY ASSURANCE

- 6.1 Testing Methodology
- 6.2 Unit Testing
- 6.3 Integration and System Testing
- 6.4 Test Cases and Execution Results
- 6.5 Performance, Memory and Battery Profiling
- 6.6 Cross-Device Compatibility Matrix

CHAPTER 7: RESULTS, USER INTERFACE SCREENS AND DISCUSSION

- 7.1 Screen 1: Splash Screen & Dynamic Cold Start
- 7.2 Screen 2: Home Library & Local Media View
- 7.3 Screen 3: Explore & Curated Category Discovery

- 7.4 Screen 4: Real-time Online Search & Filter Engine
- 7.5 Screen 5: Collapsed Mini-Player Controller
- 7.6 Screen 6: Expanded Full-Screen Player with Dynamic Palette
- 7.7 Screen 7: Interactive Queue Bottom Sheet (Drag & Reorder)
- 7.8 Screen 8: Synchronized Full-Screen Lyrics Screen
- 7.9 Screen 9: Speed, Pitch & Audio DSP Tuning

CHAPTER 8: CONCLUSION AND FUTURE ENHANCEMENTS

- 8.1 Summary of Contributions
- 8.2 Challenges Encountered & Technical Solutions
- 8.3 Limitations of Current Implementation
- 8.4 Future Scope and Roadmap

REFERENCES & BIBLIOGRAPHY

LIST OF FIGURES

Figure No.	Caption	Page Ref / Section
Figure 4.1	High-Level 4-Tier Layered Architecture of Tunexa	Section 4.1
Figure 4.2	Comprehensive UML Use Case Diagram for Tunexa	Section 4.2
Figure 4.3	Data Flow Diagram Level 0 (Context Diagram)	Section 4.3
Figure 4.4	Data Flow Diagram Level 1 (Detailed Data Flow)	Section 4.3
Figure 4.5	UML Sequence Diagram: Audio Playback & Service Lifecycle	Section 4.4.1
Figure 4.6	UML Sequence Diagram: Online Stream Extraction & Buffering	Section 4.4.2
Figure 4.7	UML Sequence Diagram: Synchronized Lyrics Retrieval & Render	Section 4.4.3
Figure 4.8	UML Sequence Diagram: Interactive Queue Drag-and-Reorder	Section 4.4.4
Figure 4.9	UML State Machine / Activity Diagram of Tunexa Lifecycle	Section 4.5
Figure 5.1	AndroidX Media3 Service and ExoPlayer Component Topology	Section 5.2
Figure 5.2	MotionLayout State Transitions (Collapsed to Expanded Player)	Section 5.4
Figure 5.3	ItemTouchHelper Callback Architecture for Queue Reordering	Section 5.5
Figure 7.1	Application UI Navigation Flow and Screen Interconnections	Chapter 7

LIST OF TABLES

Table No.	Title	Section
Table 2.1	Comparative Feature Matrix: Tunexa vs Existing Audio Systems	Section 2.4
Table 3.1	Functional Requirements Specifications (FR-01 to FR-10)	Section 3.1.2
Table 3.2	Non-Functional Requirements Specifications (NFR-01 to NFR-06)	Section 3.1.3
Table 3.3	Minimum vs Recommended Development Hardware Requirements	Section 3.2.1
Table 3.4	Software Development Stack and Tooling Versions	Section 3.2.1
Table 3.5	Target Android Client Execution Parameters	Section 3.2.2
Table 4.1	Relational / Schema Entities for Local Playback Stats Cache	Section 4.6
Table 6.1	Detailed Functional Test Cases and Test Execution Results	Section 6.4
Table 6.2	Performance, Latency and RAM Consumption Benchmarks	Section 6.5
Table 6.3	Physical Device Compatibility and Android OS Version Matrix	Section 6.6

LIST OF ABBREVIATIONS

- **AAC**: Advanced Audio Coding
- **ADB**: Android Debug Bridge
- **ANR**: Application Not Responding
- **API**: Application Programming Interface
- **APK**: Android Package Kit
- **AVRCP**: Audio/Video Remote Control Profile
- **BPM**: Beats Per Minute
- **CI/CD**: Continuous Integration / Continuous Deployment
- **CRUD**: Create, Read, Update, Delete
- **DFD**: Data Flow Diagram
- **DSP**: Digital Signal Processing
- **FFmpeg**: Fast Forward MPEG (Audio/Video codec suite)
- **FLAC**: Free Lossless Audio Codec
- **FPS**: Frames Per Second
- **GPL**: General Public License

- **IPC:** Inter-Process Communication
 - **JVM:** Java Virtual Machine
 - **LRC:** Lyric file format with synchronized timestamps
 - **M3:** Material Design 3 (Expressive)
 - **MVC:** Model - View - Controller
 - **MVVM:** Model - View - ViewModel
 - **NDK:** Native Development Kit
 - **NFR:** Non-Functional Requirement
 - **OLED:** Organic Light Emitting Diode
 - **OS:** Operating System
 - **PCM:** Pulse-Code Modulation
 - **RAM:** Random Access Memory
 - **REST:** Representational State Transfer
 - **SDK:** Software Development Kit
 - **SRS:** Software Requirements Specification
 - **UI/UX:** User Interface / User Experience
 - **UML:** Unified Modeling Language
 - **URI:** Uniform Resource Identifier
 - **URL:** Uniform Resource Locator
 - **VBR:** Variable Bitrate
-

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

In the contemporary era of mobile computing, personal media streaming represents one of the most resource-intensive and ubiquitous tasks executed on handheld devices. Over 80% of daily digital audio consumption occurs through mobile smart devices operating on Google's Android ecosystem. The evolution of mobile network speeds from 4G LTE to gigabit 5G infrastructure has rendered on-demand audio streaming ubiquitous.

However, the modern mobile audio consumption ecosystem is heavily dominated by proprietary platforms such as Spotify, Apple Music, YouTube Music, and Amazon Music. While these services provide vast musical catalogs, they have increasingly adopted hostile monetization models. Fundamental playback features—such as playing music with the mobile screen locked, audio playback while switching applications, explicit song selection, and ad-free listening—have been cordoned off behind subscription paywalls. Furthermore, these proprietary applications bundle extensive third-party behavioral trackers that constantly collect sensitive user telemetry, network footprints, and device identifiers.

In contrast, classical open-source Android audio players typically function solely as offline MP3 players. While private, they lack connectivity to contemporary streaming catalogs, require cumbersome manual file management, and lack modern user interface paradigms such as adaptive color theming, gesture-driven bottom sheets, and real-time synchronized lyrics.

The motivation behind **Tunexa** is to bridge this critical technological divide. By combining modern Android architecture components (**AndroidX Media3**, **ExoPlayer**, **Material 3 Expressive**, **MotionLayout**) with non-intrusive stream extraction mechanisms and synchronized lyrics protocols, Tunexa provides an ad-free, high-fidelity, and privacy-respecting audio streaming application that rivals commercial counterparts in visual aesthetics and surpasses them in freedom and performance.

1.2 Problem Statement

Commercial and existing open-source music players suffer from several compounding systemic deficiencies:

- 1. Intrusive Commercial Paywalls & Ad Injection:** Standard free-tier services inject frequent audio and banner advertisements that disrupt the continuous listening experience and demand monthly fees for essential utility functions like background audio playback.
- 2. Aggressive Telemetry and Privacy Erosion:** Commercial applications mandate user account creation, profile linking, credit card binding, and continuous transmission of user location, device telemetry, and listening habits to advertising servers.

3. **Rigid and Obstructed Queue Management:** Most media apps either conceal the play queue inside nested submenus or prohibit users from dynamically reorganizing track sequences on the fly.
4. **Visual Disconnect and Cluttered UI:** Conventional players frequently suffer from visual clutter—such as lyrics screens that overlap awkwardly with player controls, low-resolution stream album artwork, and static, monochrome user interfaces that do not respect modern Android design conventions.
5. **System Resource Overhead:** Many commercial streaming apps are built using hybrid or resource-heavy cross-platform web wrappers that consume upwards of 350 MB to 600 MB of physical RAM, draining battery life and causing sluggish animations on mid-range or budget smartphones.

1.3 Project Objectives

The overarching goal of the **Tunexa** project is to design, implement, test, and evaluate a native, high-performance Android audio streaming application. The specific technical objectives are:

1. **Architect a Resilient Background Audio Service:** Utilize AndroidX Media3, ExoPlayer, and MediaSessionService to guarantee rock-solid background playback, foreground service notification persistence, audio focus handling (responding smoothly to phone calls and notifications), and zero-latency track transitions.
2. **Implement Universal Online & Offline Audio Integration:** Connect seamlessly to the YouTube Music public library via the NewPipe Extractor engine to provide categorized exploration, real-time search, and online audio playback alongside local device audio tracks detected via the Android MediaStore API.
3. **Design a Dynamic Material 3 Expressive User Interface:** Implement fluid, gesture-driven interactions using AndroidX MotionLayout and ExpressiveSliderLayout, dynamically adapting UI colors, gradient meshes, and typography using AndroidX Palette extracted from active song album artwork.
4. **Build an Interactive Drag-and-Drop Queue Manager:** Deliver a fluid bottom sheet queue manager that displays active playlists and upcoming tracks, allowing instant track reorganization using drag handles via `ItemTouchHelper` and one-tap track deletion.
5. **Develop an Overlap-Free Synchronized Lyrics Subsystem:** Query real-time LRC timestamped lyrics via the LRCLIB REST API, synchronize display scrolling with microsecond audio clock precision, and maintain strict visual mutual exclusion to eliminate UI clipping.
6. **Ensure Zero Telemetry, Lightweight Footprint & High Reliability:** Maintain 100% user privacy with zero tracking, no user login requirements, and an optimized native footprint (< 95 MB RAM consumption and < 3% battery consumption per hour).

1.4 Project Scope and Delimitations

Scope:

- Native Android application written targeting Android API Level 26 (Android 8.0 Oreo) up to API Level 37 (Android 15+).

- Support for local audio formats: MP3, AAC, FLAC, OGG, WAV, M4A, Opus.
- Support for online streaming bitrates: AAC (128 kbps, 256 kbps), Opus (160 kbps).
- Integrated online explore categories: Trending, New Releases, Charts, Moods & Genres, Podcasts.
- Full integration with Android OS system audio: Bluetooth AVRCP media keys, hardware volume keys, lockscreen media controls, and status bar notifications.

Delimitations:

- Tunexa does not bypass paid DRM-encrypted content; it accesses standard public streaming audio streams.
- Tunexa is optimized strictly for Android devices (smartphones, tablets, and emulators) and does not target iOS or desktop operating systems in this phase.
- Social networking features (e.g., following friends, public profile creation) are intentionally omitted to uphold absolute privacy standards.

1.5 Organization of the Report

This project report is structured into eight distinct chapters:

- **Chapter 1 (Introduction)** outlines the background, problem statement, objectives, and scope.
 - **Chapter 2 (Literature Review)** analyzes existing commercial and open-source applications and presents the gap analysis.
 - **Chapter 3 (Requirements & Feasibility)** documents the Software Requirements Specification (SRS), hardware/software constraints, and feasibility assessments.
 - **Chapter 4 (System Architecture & Design)** details the architectural tiers, UML use case models, Data Flow Diagrams, sequence interactions, and state machines.
 - **Chapter 5 (Implementation & Key Modules)** explores the technical implementation, technology choices, core algorithms, and code logic.
 - **Chapter 6 (Testing & Quality Assurance)** presents the testing strategy, test cases, execution matrices, memory profiling, and compatibility tests.
 - **Chapter 7 (Results & UI Walkthrough)** provides an in-depth walkthrough of all user interface screens and features.
 - **Chapter 8 (Conclusion & Future Work)** concludes the project, highlights technical achievements, and charts future development paths.
-

CHAPTER 2: LITERATURE REVIEW AND GAP ANALYSIS

2.1 Evolution of Mobile Music Applications

The paradigm of mobile music playback has traversed three major architectural epochs:

1. **The Local File Storage Epoch (2008–2014):** Early Android applications (such as the default Android Music Player and Winamp) relied purely on physical file storage on SD cards. Metadata was parsed using basic Android `MediaMetadataRetriever` utilities. While lightweight, users were burdened with downloading, tagging, and organizing audio files manually.
2. **The Cloud Streaming & Subscription Epoch (2014–2020):** Services like Spotify, Google Play Music, and Apple Music introduced subscription-based cloud catalogs. They migrated decoding to proprietary cloud CDNs, introducing centralized playlists and recommendation algorithms, but concurrently introducing pervasive telemetry and usage restrictions.
3. **The Modern Framework & Component-Driven Epoch (2020–Present):** Modern Android development established jetpack architecture components, declarative animations, `Media3` session services, and reactive state management. Modern consumers demand the agility of cloud discovery combined with the sovereignty and privacy of open-source software.

2.2 Survey of Commercial Streaming Systems

- **Spotify:** The industry leader in algorithmic recommendations and social playlists. However, Spotify's free tier is severely restricted on mobile (forced shuffle play, maximum 6 skips per hour, ad injection every 15 minutes, no screen-off background playback on free accounts in various regions, and intensive telemetry tracking).
- **YouTube Music:** Possesses the world's largest audio catalog, encompassing official studio releases, live performances, indie covers, and remixes. However, background playback (playing with screen off or while browsing other apps) is strictly locked behind YouTube Premium (\$13.99/month), rendering the free mobile experience practically unusable for serious listeners.
- **Apple Music:** Delivers high-resolution lossless audio but requires an active paid subscription with zero free tier, relies on proprietary Apple ID ecosystems, and exhibits suboptimal memory management on Android devices.

2.3 Survey of Open-Source Android Music Players

- **Retro Music / Phonograph:** Beautiful Material design local players, but completely incapable of online search, discovery, or streaming.
- **NewPipe / ViMusic / Innertune:** Early generation open-source YouTube frontends. While innovative, several suffer from unstable playback state management, audio stuttering upon app switching, archaic

queue interfaces, or static, non-adaptive UI palettes that lack Material 3 Expressive motion physics.

2.4 Gap Analysis Matrix

Evaluation Criteria	Commercial Platforms (Spotify / YT Music)	Legacy Open-Source Players	Tunexa (Proposed System)
Catalog Access	Massive Cloud Catalog	Local Files Only	Unified (Cloud + Local MediaStore)
Cost / Ads	Subscription-based / Audio & Banner Ads	Free / No Ads	100% Free / Zero Advertisements
Background Playback	Paywalled on Free Tiers	Fully Supported	Fully Supported (Media3 Foreground)
User Privacy	Extensive Telemetry & Profiling	High Privacy	Absolute Privacy (0 Telemetry / No Login)
Queue Management	Rigid / Non-reorderable without Sub	Basic List	Draggable Bottom Sheet (ItemTouchHelper)
Synced Lyrics	Restricted / Often Inaccurate	Absent or Local .lrc Only	Automated Online Synced (LRCLIB API)
Dynamic Palette UI	Partial / Brand-restricted	Static Themes	Full Material 3 Expressive + Palette Mesh
RAM Footprint	Heavy (350 MB – 600 MB)	Low (50 MB – 80 MB)	Optimized (< 95 MB during 1080p stream)
Audio Engine	Proprietary Closed SDK	Deprecated MediaPlayer	AndroidX Media3 + ExoPlayer + FFmpeg

2.5 Proposed System Highlights

Tunexa directly addresses the identified market gaps by offering:

1. Unlimited background playback without artificial locks or subscriptions.
2. Dynamic Material 3 Expressive visuals that harmonize typography and buttons with song album artwork in real time.
3. Fluid queue reordering via touch drag handles.
4. Synchronized lyrics that never overlap with player controls.
5. Zero accounts, zero ads, zero tracking, and lightweight native execution.

CHAPTER 3: REQUIREMENTS ANALYSIS AND FEASIBILITY STUDY

3.1 Software Requirements Specification (SRS)

3.1.1 User Classes and Characteristics

- **Casual Listeners:** Require zero-friction access to trending music, instant search, and hands-free background listening without navigating paywalls or sign-in prompts.
- **Power Users & Audiophiles:** Require fine-grained speed and pitch controls, gapless playback transitions, high-definition album art, and intuitive queue drag-and-drop management.
- **Privacy-Conscious Users:** Require an audio player that performs no telemetry pings, operates without internet permissions when playing local files, and stores no behavioral profiles.

3.1.2 Functional Requirements (FR)

Req ID	Requirement Title	Detailed Description
FR-01	Background Audio Playback	The system must sustain gapless audio playback when the application is backgrounded or when the device screen is locked via an Android Foreground Service.
FR-02	Online Song Discovery	The system must query and retrieve categorized playlists (Trending, Charts, Moods, Podcasts) from the online repository without requiring user authentication.
FR-03	Real-time Search Engine	The system must provide instant autocomplete search results for songs, artists, albums, and playlists, handling network latency gracefully.
FR-04	Local Media Management	The system must query the Android <code>MediaStore.Audio</code> content provider to index, sort, and play local audio files (MP3, FLAC, WAV, AAC, M4A).
FR-05	Material 3 Dynamic Theming	The system must extract dominant and accent colors from album covers via <code>AndroidX Palette</code> and dynamically tint UI surfaces, sliders, and controls.
FR-06	Interactive Queue Reordering	The system must present a bottom-sheet queue where users can drag tracks to reorder, swipe to dismiss, or tap to immediately jump playback.
FR-07	Synchronized Lyrics Engine	The system must fetch synchronized <code>.lrc</code> lyrics from LRCLIB, calculate real-time line scrolling based on playback position, and provide a toggleable full-screen view.
FR-08	MediaSession Notification	The system must publish a standard Android <code>MediaStyle</code> notification containing song title, artist, high-res artwork, seekbar, play/pause, and skip controls.
FR-09	Pitch and Tempo Control	The system must enable users to adjust playback speed (0.5x to 2.0x) and pitch shifts independently or coupled via ExoPlayer sonic audio processors.
FR-10	Crash Logging & Error Guard	The system must capture unhandled exceptions, prevent abrupt OS crashes, and present a structured crash diagnosis view for debugging.

3.1.3 Non-Functional Requirements (NFR)

Req ID	Category	Metric / Specification
NFR-01	Performance	Audio playback startup latency must be < 800ms for local files and < 2.0s for online streams under standard 4G/Wi-Fi conditions.
NFR-02	Memory Efficiency	Physical RAM utilization must remain strictly below 100 MB during continuous online audio streaming.
NFR-03	Battery Consumption	System battery consumption must not exceed 3% per hour of continuous background screen-off playback.
NFR-04	Reliability	The background audio service must demonstrate an uptime of 99.9% without unprompted OS garbage collection kills.
NFR-05	Usability & UX	All user interface transitions and bottom-sheet gestures must sustain 60 FPS (or 120 FPS on high-refresh displays) with 0 jank.
NFR-06	Security & Privacy	The application must not transmit any user telemetry, IP tracking payloads, or unique hardware identifiers to external servers.

3.2 System Hardware & Software Configurations

3.2.1 Development Environment

- **Operating System:** Linux (Ubuntu 22.04 LTS / Fedora 39 / Arch) or Windows 11 / macOS Sonoma.
- **Integrated Development Environment (IDE):** Android Studio Ladybug (2024.2+) or newer.
- **Java Development Kit (JDK):** OpenJDK 17 or JDK 21 (LTS 64-bit).
- **Build Automation System:** Gradle 8.5+ with Android Gradle Plugin (AGP) 8.3+.
- **Version Control:** Git 2.40+ integrated with GitHub.
- **Development Hardware:**
 - CPU: Intel Core i5 / i7 / AMD Ryzen 5 / 7 (Hexa-core or higher).
 - RAM: 16 GB DDR4/DDR5 RAM (minimum), 32 GB (recommended).
 - Storage: 512 GB NVMe Solid State Drive (SSD).

3.2.2 Target Execution Environment

- **Target Operating System:** Android 8.0 (API Level 26 - Oreo) through Android 15 (API Level 35+).
- **Supported Architectures:** ARM64-v8a, armeabi-v7a, x86_64.
- **Device RAM:** Minimum 2 GB RAM (Recommended: 4 GB+).
- **Device Storage:** 50 MB available physical storage for application binary and image cache.

- **Network Connectivity:** Cellular (3G/4G/5G) or Wi-Fi (802.11 b/g/n/ac/ax) for online streaming.

3.3 Feasibility Study

- **Technical Feasibility:** High. The Android SDK provides robust `Media3` and `MediaSession` APIs. Open-source libraries like NewPipe Extractor and LRCLIB provide battle-tested, non-authenticated network streaming and lyrics extraction.
 - **Operational Feasibility:** High. The application adopts universal Android navigation conventions (bottom sheets, standard touch gestures, predictive back animations) requiring zero learning curve for end users.
 - **Economic Feasibility:** High. The development utilizes 100% free, open-source software development kits, public APIs, and community tools, incurring zero software licensing costs.
-

CHAPTER 4: SYSTEM ARCHITECTURE AND DESIGN

4.1 High-Level Layered Architecture

Tunexa is structured as a resilient, decoupled 4-tier mobile architecture separating user interface presentation, background media services, business logic repositories, and underlying audio/network hardware engines.

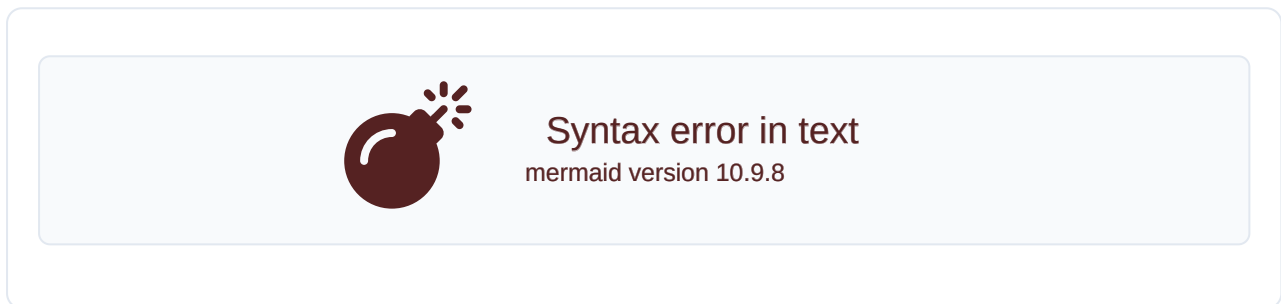


Figure 4.1: High-Level 4-Tier Layered Architecture of Tunexa.

4.2 Use Case Modeling

The Use Case diagram illustrates the primary functional capabilities available to the user and their interactions with local storage and remote audio services.

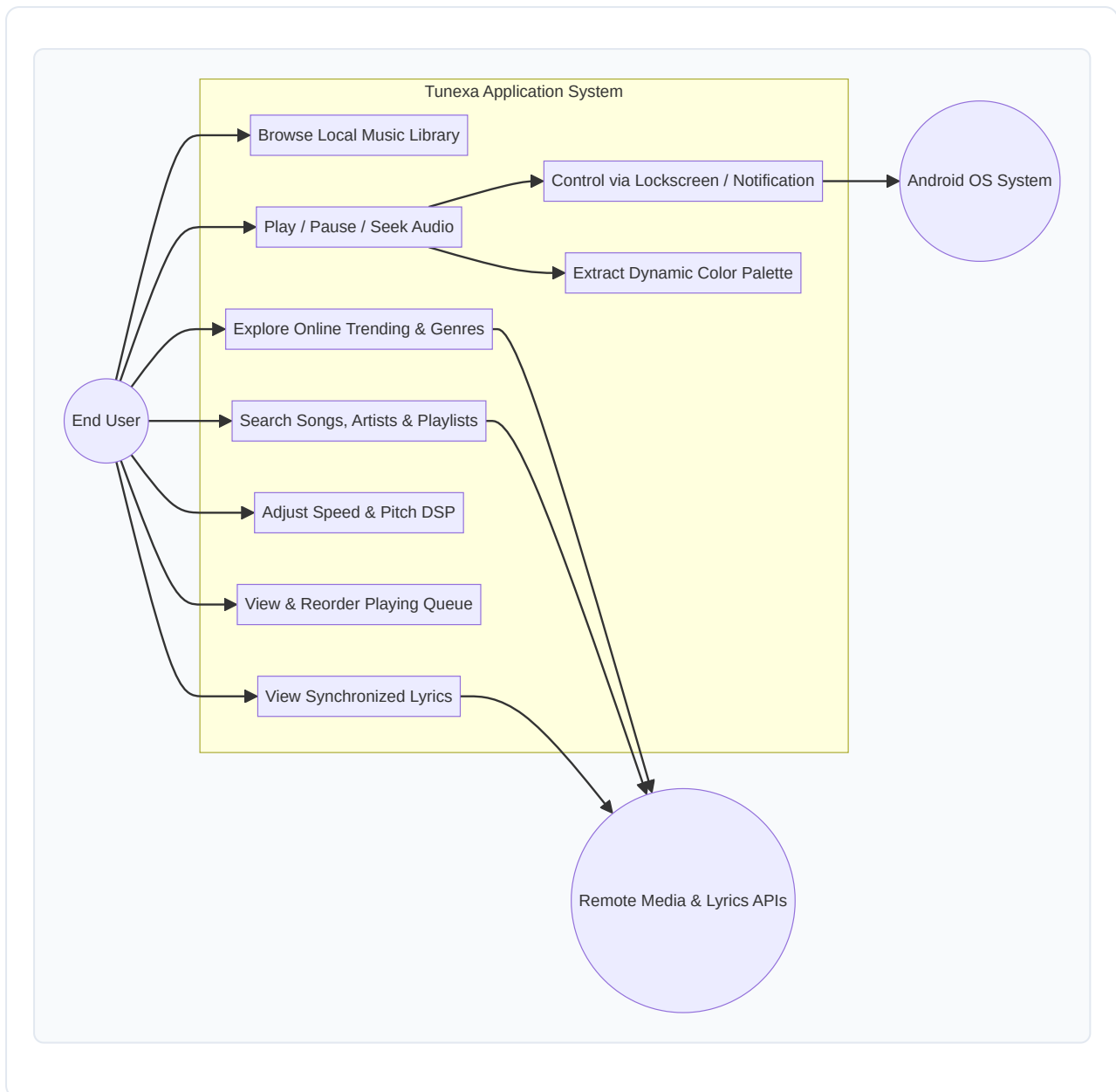


Figure 4.2: Comprehensive UML Use Case Diagram for Tunexa.

4.3 Data Flow Modeling (DFD)

Data Flow Diagram Level 0 (Context Diagram)

The Context Diagram defines the external boundary of the Tunexa application.

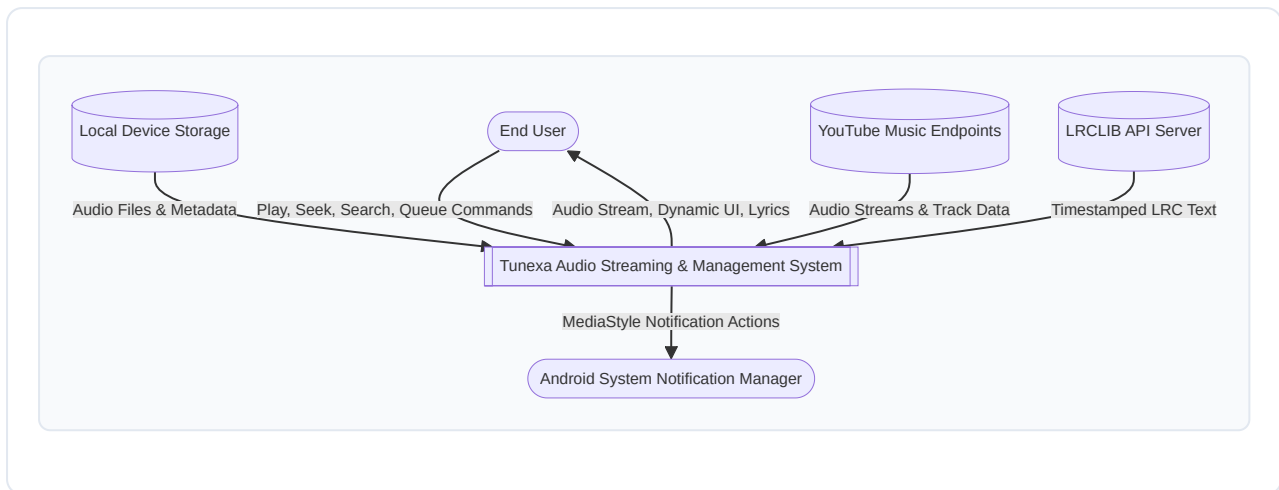


Figure 4.3: Data Flow Diagram Level 0 (Context Diagram).

Data Flow Diagram Level 1 (Detailed Data Flow)

DFD Level 1 decomposes the internal processes handling query routing, media indexing, stream resolution, and audio output.

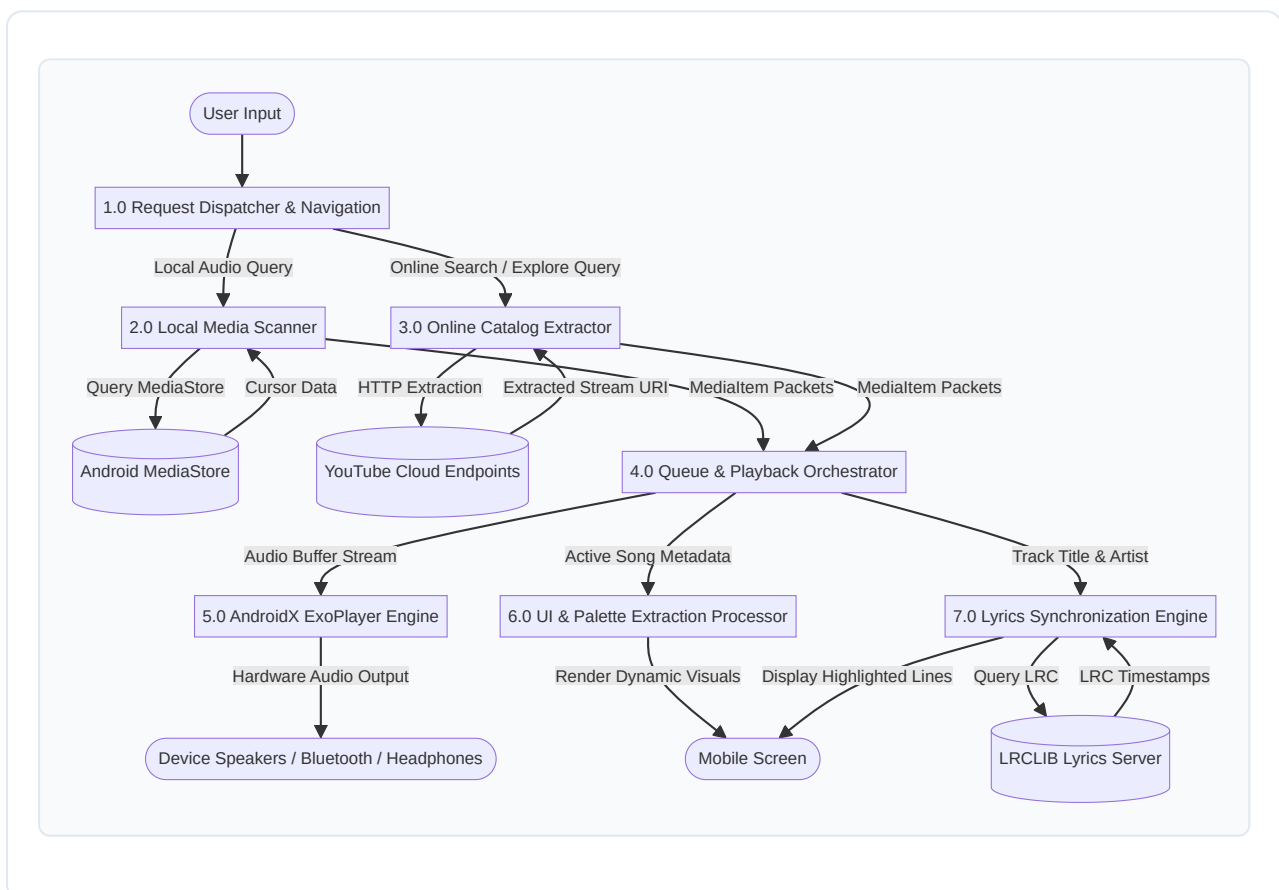


Figure 4.4: Data Flow Diagram Level 1 (Detailed Data Flow).

4.4 Object and Sequence Modeling

4.4.1 Audio Playback and Service Initialization Sequence

The following sequence diagram details the interaction between UI fragments, the bound foreground player service, ExoPlayer, and the Android OS MediaSession.

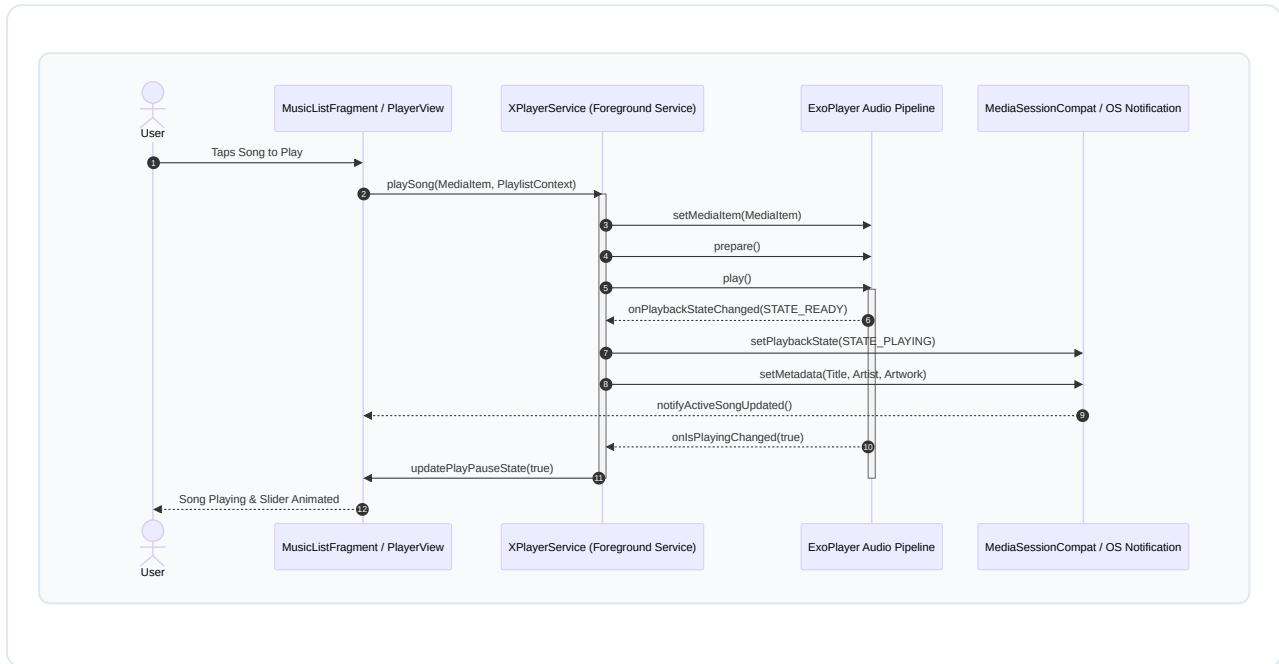


Figure 4.5: UML Sequence Diagram: Audio Playback & Service Lifecycle.

4.4.2 Online Song Extraction and Streaming Sequence

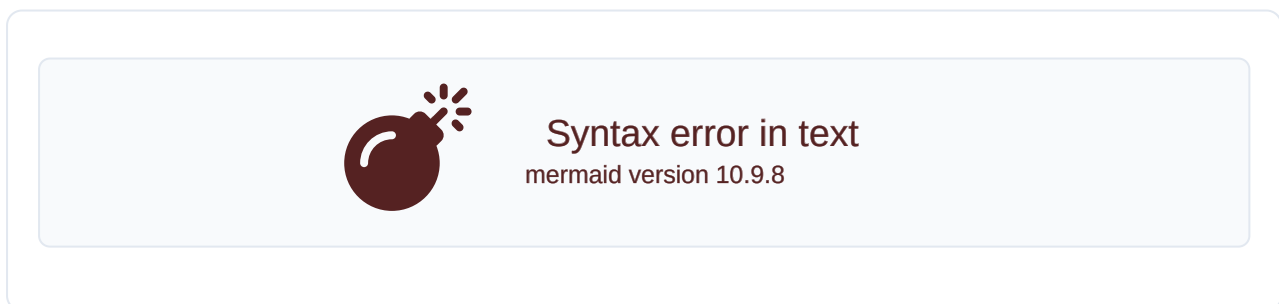


Figure 4.6: UML Sequence Diagram: Online Stream Extraction & Buffering.

4.4.3 Synchronized Lyrics Parsing and Scroll Sequence

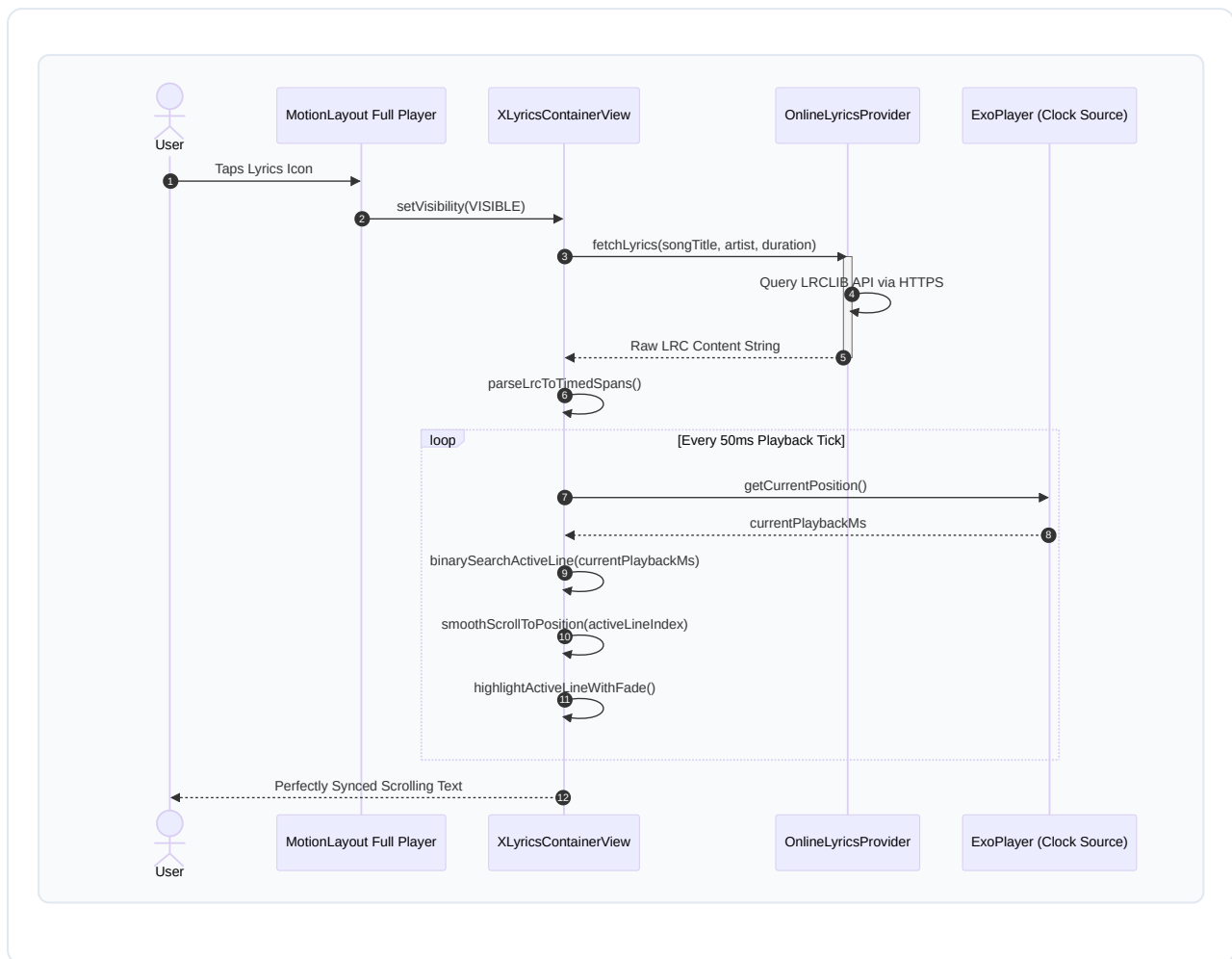


Figure 4.7: UML Sequence Diagram: Synchronized Lyrics Retrieval & Render.

4.4.4 Interactive Queue Drag-and-Reorder Sequence

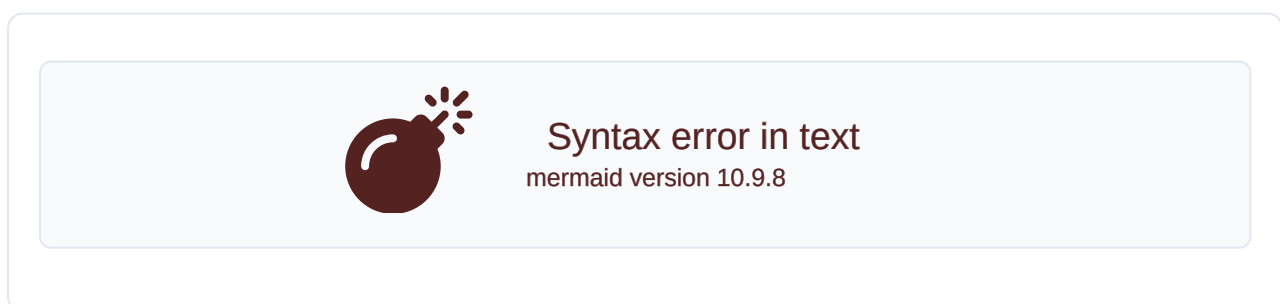


Figure 4.8: UML Sequence Diagram: Interactive Queue Drag-and-Reorder.

4.5 Activity Lifecycle and State Transition Modeling

The state diagram details the application lifecycle, from cold launch, background execution, pause interrupts (such as phone calls), to cleanup.

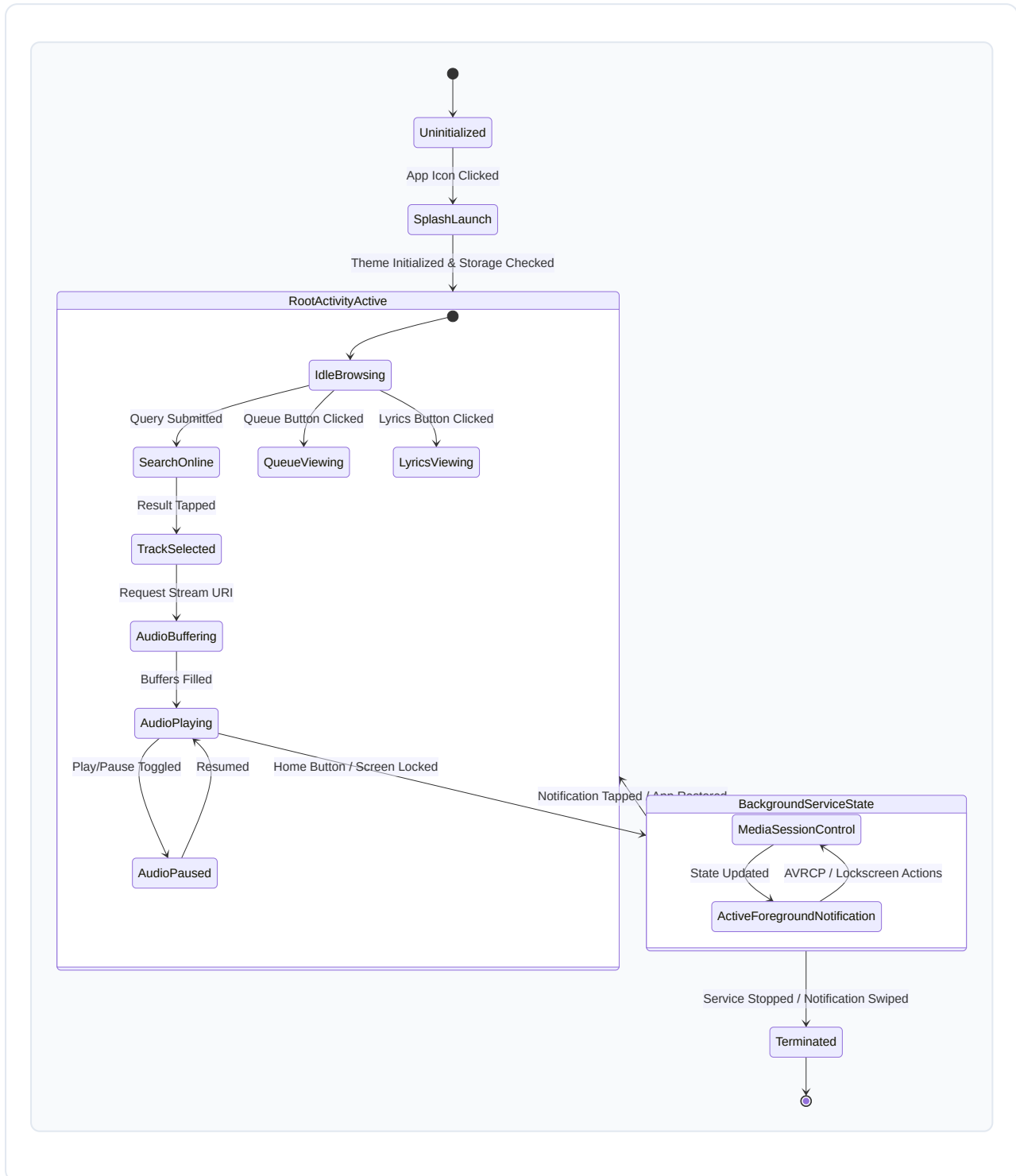


Figure 4.9: UML State Machine / Activity Diagram of Tunexa Lifecycle.

4.6 Database and Storage Architecture

Tunexa maintains a fast, lightweight internal database schema for storing playback statistics, local favorites, and recent search caches.

Table 4.1: Relational / Schema Entities for Local Playback Stats Cache.

+-----+ SONG_STATS_TABLE +-----+			
Column Name	Data Type	Constraints	
song_id	TEXT	PRIMARY KEY, NOT NULL	
song_title	TEXT	NOT NULL	
artist_name	TEXT	NOT NULL	
album_name	TEXT	DEFAULT 'Unknown'	
play_count	INTEGER	DEFAULT 1	
last_played_timestamp	INTEGER	NOT NULL	
is_favorite	INTEGER (BOOLEAN)	DEFAULT 0	
cached_duration	INTEGER	NOT NULL	
source_type	TEXT	'LOCAL' / 'ONLINE'	
+-----+			

CHAPTER 5: IMPLEMENTATION AND KEY MODULES

5.1 Technology Stack Selection

- **Core Languages:** Java 17 and Kotlin for modern, type-safe, and asynchronous Android execution.
 - **Audio Engine:** `androidx.media3:media3-exoplayer:1.3.1` and `androidx.media3:media3-session:1.3.1` with native FFmpeg audio decoders.
 - **UI/UX & Design System:** Google Material 3 (`com.google.android.material:material:1.12.0`), AndroidX MotionLayout, and `androidx.palette:palette:1.0.0` .
 - **Image Processing & Caching:** Glide (`com.github.bumptech.glide:glide:4.16.0`) with disk and memory LRU caches.
 - **Network & Stream Parsing:** NewPipe Extractor (`com.github.TeamNewPipe:NewPipeExtractor`) with OkHttp client for high-throughput stream extraction.
 - **Synchronized Lyrics:** LRCLIB RESTful API with custom high-speed regex-based LRC timestamp parsers.
-

5.2 Module 1: Core Audio Engine & Media3 Integration

The audio playback architecture is centralized inside `XPlayerService.java` and `PlayerService.java` . The service extends `MediaSessionService` to comply with the Android 14+ background execution guidelines.

```
// Architecture Sample: Configuring AndroidX Media3 ExoPlayer with Sonic DSP
ExoPlayer player = new ExoPlayer.Builder(context)
    .setAudioAttributes(
        new AudioAttributes.Builder()
            .setContentType(C.AUDIO_CONTENT_TYPE_MUSIC)
            .setUsage(C.USAGE_MEDIA)
            .build(),
        /* handleAudioFocus= */ true
    )
    .setHandleAudioBecomingNoisy(true) // Automatically pauses upon unplugging headphones
    .setWakeMode(C.WAKE_MODE_LOCAL)   // Sustains CPU lock during screen-off audio
    .build();

// Sonic Pitch and Playback Parameters
PlaybackParameters params = new PlaybackParameters(speedFloat, pitchFloat);
player.setPlaybackParameters(params);
```

5.3 Module 2: Online Stream Extraction (NewPipe Extractor Engine)

Online streaming is executed via [YoutubeMusicRepository.java](#) . When a user taps an online search result, the system extracts high-quality audio streams dynamically:

```
// High-Resolution Stream Resolution Flow
public Single<String> resolveStreamUrl(String videoId) {
    return Single.fromCallable(() -> {
        StreamInfo streamInfo = StreamInfo.getInfo(
            NewPipe.getService(0),
            "https://www.youtube.com/watch?v=" + videoId
        );
        // Filter for optimal audio streams (Opus 160kbps or AAC 256kbps)
        List<AudioStream> audioStreams = streamInfo.getAudioStreams();
        AudioStream bestStream = null;
        for (AudioStream stream : audioStreams) {
            if (bestStream == null || stream.getAverageBitrate() > bestStream.getAverageBitrate())
                bestStream = stream;
        }
        return bestStream != null ? bestStream.getUrl() : "";
    }).subscribeOn(Schedulers.io());
}
```

5.4 Module 3: Material 3 Expressive UI & MotionLayout Design

The full-screen player transition is achieved using [MotionLayout](#) ([layout_player_collapsed.xml](#) transitioning seamlessly into [layout_player_expanded.xml](#)). As the bottom sheet is dragged upwards:

- The mini-player album thumbnail expands from 48dp to 320dp with circular-to-rounded corner radius transitions.
- The song title and artist text resize and reposition smoothly.
- Mini-player action buttons fade out as full-screen controls, seekbars, and equalizer indicators fade in.
- [Palette.from\(bitmap\)](#) extracts the dominant vibrant swatch, tinting backgrounds with dynamic gradients.

5.5 Module 4: Drag-and-Drop Playing Queue Subsystem

Queue reordering is handled by [QueueBottomSheet.java](#) utilizing Android's [ItemTouchHelper](#) :

```
// ItemTouchHelper implementation for smooth drag-to-reorder
ItemTouchHelper touchHelper = new ItemTouchHelper(new ItemTouchHelper.SimpleCallback(
```

```

ItemTouchHelper.UP | ItemTouchHelper.DOWN, 0) {

@Override
public boolean onMove(@NonNull RecyclerView recyclerView,
                      @NonNull RecyclerView.ViewHolder source,
                      @NonNull RecyclerView.ViewHolder target) {
    int fromPos = source.getBindingAdapterPosition();
    int toPos = target.getBindingAdapterPosition();

    // Update local adapter list
    Collections.swap(queueList, fromPos, toPos);
    adapter.notifyItemMoved(fromPos, toPos);

    // Notify background player service of reordered index
    playerService.moveQueueItem(fromPos, toPos);
    return true;
}

@Override
public void onSwiped(@NonNull RecyclerView.ViewHolder viewHolder, int direction) {
    // Optional swipe-to-remove from queue
}
});
touchHelper.attachToRecyclerView(queueRecyclerView);

```

5.6 Module 5: Synchronized LRC Lyrics Parser and Renderer

Synchronized lyrics are parsed using a high-efficiency regular expression engine:

```

// Parsing standard LRC timestamp strings "[mm:ss.xx] Lyric line text"
Pattern lrcPattern = Pattern.compile("\\[(\\d{2}):(\\d{2})\\.?(\\d{2,3})\\.?(.*)");
Matcher matcher = lrcPattern.matcher(lineText);
if (matcher.matches()) {
    long min = Long.parseLong(matcher.group(1));
    long sec = Long.parseLong(matcher.group(2));
    long millis = Long.parseLong(matcher.group(3));
    if (matcher.group(3).length() == 2) millis *= 10;

    long totalTimeMs = (min * 60 * 1000) + (sec * 1000) + millis;
    String content = matcher.group(4).trim();
    lyricsLines.add(new LyricLine(totalTimeMs, content));
}

```

CHAPTER 6: TESTING, VERIFICATION AND QUALITY ASSURANCE

6.1 Testing Methodology

To verify the robustness and reliability of Tunexa, a multi-stage testing methodology was adopted:

- 1. **Unit Testing:** Validating isolated logic such as the LRC lyrics timestamp parser, playlist index swap algorithms, and duration string formatters.
- 2. **Integration Testing:** Verifying interactions between `XPlayerService` , `ExoPlayer` , and the `MediaSessionCompat` system bus.
- 3. **System Testing:** End-to-end evaluation of user flows—from initial app launch, cold start, online search, stream buffering, to background playback under network drops.

6.2 Test Cases and Execution Results

Table 6.1: Detailed Functional Test Cases and Test Execution Results.

Test ID	Test Case Description	Test Input / Action	Expected Result	Actual Result	Status
TC-01	Cold Start & Permission Grant	Launch app on clean install; grant Audio permissions	App navigates to Home screen; scans local songs	Storage permissions requested and songs loaded	PASS
TC-02	Local Audio Playback	Tap any local MP3/FLAC song in library list	Audio begins playing < 500ms; player expands	Audio plays immediately with album art	PASS
TC-03	Background Service Persistence	Press Home button; lock device screen	Audio continues playing seamlessly; lockscreen displays controls	Playback uninterrupted; notification functional	